Solution: Counting Elements (easy)

Problem Statement

Given a list of integers, determine the count of numbers for which there exists another number in the list that is greater by exactly one unit.

In other words, for each number `x` in the list, if `x + 1` also exists in the list, then `x` is considered for the count.

Examples:

1. **Example 1**:
    - Input: [4, 3, 1, 5, 6]
    - Expected Output: 3
    - Justification: The numbers 4, 3, and 5 have 5, 4, and 6 respectively in the list, which are greater by exactly one unit.
2. **Example 2**:
    - Input: [7, 8, 9, 10]
    - Expected Output: 3
    - Justification: The numbers 7, 8, and 9 have 8, 9, and 10 respectively in the list, which are greater by exactly one unit.
3. **Example 3**:
    - Input: [11, 13, 15, 16]
    - Expected Output: 1
    - Justification: Only the number 15 has 16 in the list, which is greater by exactly one unit.

Solution

To solve this problem, the first step is to add all elements of the given array into a HashSet. This allows for constant time complexity (O(1)) checks for the presence of any number. After populating the HashSet, iterate through the array once more. During this iteration, for each element, check if its direct successor (the element itself plus one) is present in the HashSet. If the successor is found, increment a counter. This counter will keep track of the total number of elements that have their direct successors within the array. The final value of this counter, after completing the iteration over the array, will be the solution to this problem.

1. **Hashset Creation**: Begin by initializing an empty hashset. Traverse the list and insert each number into the hashset. This will allow for constant-time lookups to check the existence of any number in the list.
2. **Counting Valid Numbers**: Traverse the list again. For each number `x`, check if `x + 1` exists in the hashset. If it does, increment a counter. This counter will keep track of all the numbers that satisfy the given condition.
3. **Result**: After traversing the entire list, the counter will hold the total count of numbers for which there exists another number greater by exactly one unit.
4. **Return**: Return the counter as the final result.

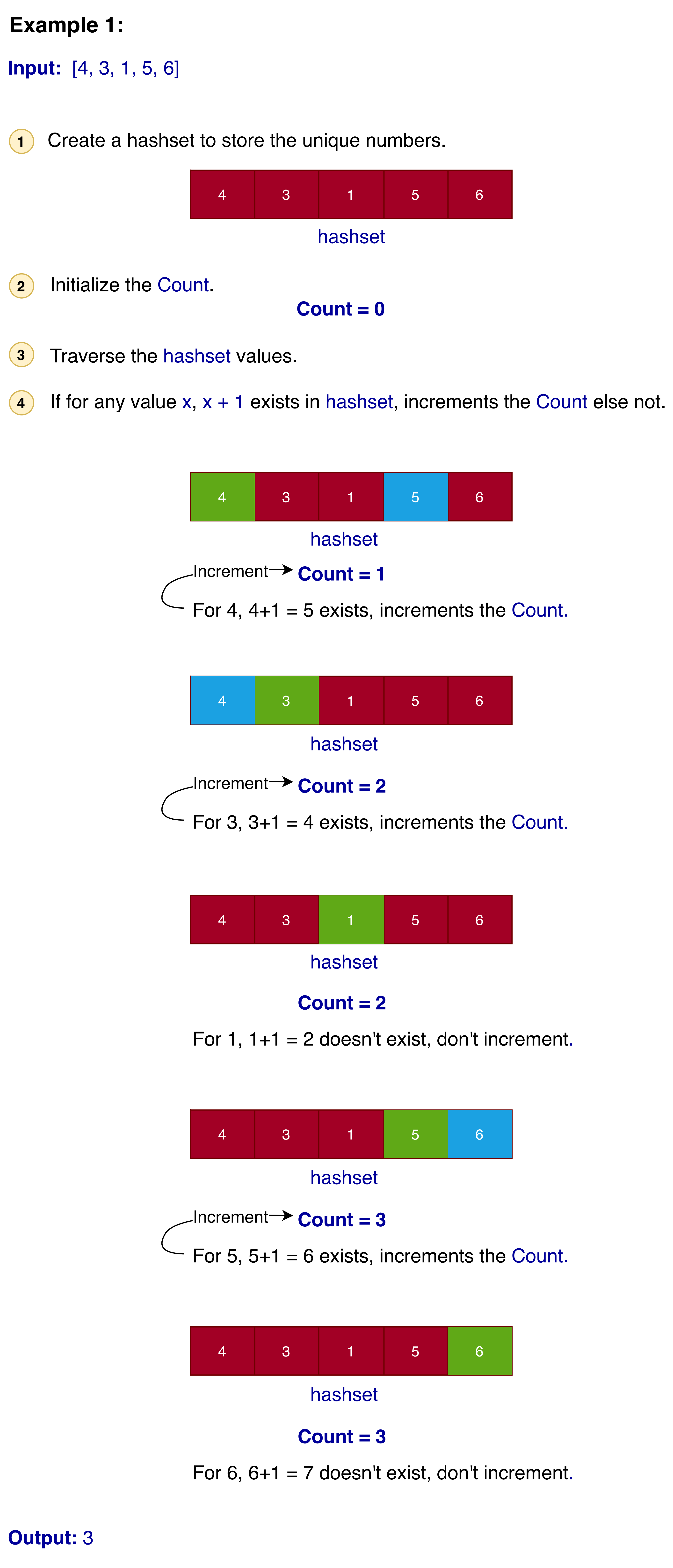
This approach is efficient because it leverages the constant-time lookup property of hashset, ensuring that the algorithm runs in linear time.

Algorithm Walkthrough:

Given the input list [4, 3, 1, 5, 6]:

- Initialize an empty hashset.
- Insert each number from the list into the hashset: {4, 3, 1, 5, 6}.
- Initialize a counter to 0.
- Traverse the list:
    - For 4, check if 5 exists in the hashset. It does, so increment the counter.
    - For 3, check if 4 exists in the hashset. It does, so increment the counter.
    - For 1, check if 2 exists in the hashset. It doesn't, so move on.
    - For 5, check if 6 exists in the hashset. It does, so increment the counter.
    - For 6, check again if 7 exists in the hashset. It still doesn't, so move on.
- The counter now holds the value 3, which is the final result.

Here is the visual representation of the algorithm:


Example 1:
Input: [4, 3, 1, 5, 6]
1 Create a hashset to store the unique numbers.
4 3 1 5 6
hashset
2 Initialize the Count.
Count = 0
3 Traverse the hashset values.
4 If for any value x, x + 1 exists in hashset, increments the Count else not.
4 3 1 5 6
hashset
Increment
Count = 1
For 4, 4+1 = 5 exists, increments the Count.
4 3 1 5 6
hashset
Increment
Count = 2
For 3, 3+1 = 4 exists, increments the Count.
4 3 1 5 6
hashset
Count = 2
For 1, 1+1 = 2 doesn't exist, don't increment.
4 3 1 5 6
hashset
Increment
Count = 3
For 5, 5+1 = 6 exists, increments the Count.
4 3 1 5 6
hashset
Count = 3
For 6, 6+1 = 7 doesn't exist, don't increment.
Output: 3

Code

Here is the code for this algorithm:

Java

```java
import java.util.HashSet;

public class Solution {
    public int countElements(int[] arr) {
        // Create a set to store unique numbers from the array
        HashSet<Integer> set = new HashSet<>();

        // Populate the set with numbers from the array
        for (int num : arr) {
            set.add(num);
        }

        int count = 0;
        // For each number in the array, check if its next consecutive number exists in
        for (int num : arr) {
            if (set.contains(num + 1)) {
                count++;
            }
        }

        return count;
```

Run Code Result | Stdout

You must run your code first

Execute

Complexity Analysis

Time Complexity: The algorithm traverses the array twice: once to populate the hashset and once to count the valid numbers. Both operations are O(n), where n is the length of the array. Therefore, the overall time complexity is O(n).

Space Complexity: The space complexity is determined by the hashset, which in the worst case will have an entry for each unique number in the array. Therefore, the space complexity is O(n), where n is the number of unique numbers in the array.

← Previous
Problem 1: Counting Elements (Easy)

Next →
Problem 2: Jewels And Stones (Easy)

☐ Mark as Completed